
Embedded Laboratory

Semester Project Report — Summer Semester 2026

Multi-Node IoT Home Automation via MQTT & Bluetooth Classic

Project Track B3+ · Networking, Dashboards & Remote Control

Name	Matr.-Nr.	Programme
Prathamesh Bhagwan Dhongade	20744016	M.Sc. Mechatronics
Yash Sachitanand Dongre	10946521	M.Sc. Mechatronics
Pankaj Dattatray Ukirde	18146362	M.Sc. EMM

Module: Embedded Laboratory
Demo (Block 4): 02 July 2026
Submission: 14 August 2026, 23:59

Contents

1	Project Overview	2
1.1	Original B3 Card vs. What Was Built	2
1.2	Core Deliverable (Demoed 02 Jul 2026)	2
2	System Architecture	2
2.1	Communication Topology	2
2.2	Hardware Nodes	3
3	MQTT Topic Architecture	3
3.1	Namespace and Broker	3
3.2	Topic Map	3
3.3	Reliability Features	4
4	Key Features	4
4.1	Physical TFT HMI with Button Navigation	5
4.2	Smooth Servo Control	5
4.3	LDR Automatic Lighting with Moving-Average Filter	5
4.4	Temperature-Triggered ESP32 Relay	5
4.5	Bidirectional LED Synchronisation (SYNC Pattern)	5
4.6	Disco Mode	5
4.7	Remote Access via Tailscale	6
5	FUXA Dashboard	6
6	Pin Assignments	6
7	Software Architecture	7
7.1	Pico W Firmware	7
7.2	ESP32 Firmware	7
8	Stretch Goals Achieved	8
9	Reflection	8
9.1	What Worked Well	8
9.2	Challenges	8
9.3	Future Work	9
10	Appendix A: Libraries Used	9
11	Appendix B: Submission Contents	9

Project Overview

This report documents a multi-node IoT home-automation system built for the Embedded Laboratory semester project. The project extends the original **B3 — Home Assistant Integration via MQTT Discovery** card into a broader demonstrator that covers all B3 core deliverables while adding a second hardware node, a Bluetooth Classic communication link, a FUXA SCADA/HMI dashboard, and remote access via Tailscale.

Original B3 Card vs. What Was Built

The B3 card required the Joy-IT XPLOER Pico W to appear as native Home Assistant entities via MQTT Discovery, covering temperature/humidity, NeoPixels, servo angle, buttons as triggers, and LWT/online-offline status. During Block 1 scoping the team extended this scope as follows:

- **Second hardware node:** An ESP32 with an I/O shield was added, connected to the Pico W via **Bluetooth Classic SPP** rather than WiFi, to demonstrate a genuine multi-node heterogeneous topology.
- **FUXA instead of Home Assistant:** The open-source FUXA SCADA/HMI (v1.3.3) replaces Home Assistant, providing an equivalent dashboard, data logging, and trend charts without requiring a running HA instance.
- **Physical TFT HMI:** A 4-button navigation interface on the 1.8 inch TFT display allows local control without any external device.
- **Remote access:** Tailscale VPN exposes the FUXA dashboard from any network without port forwarding.

All B3 deliverables are met: temp/humidity sensor, NeoPixel light entity, servo control, button triggers, retained topics, and LWT online/offline signalling.

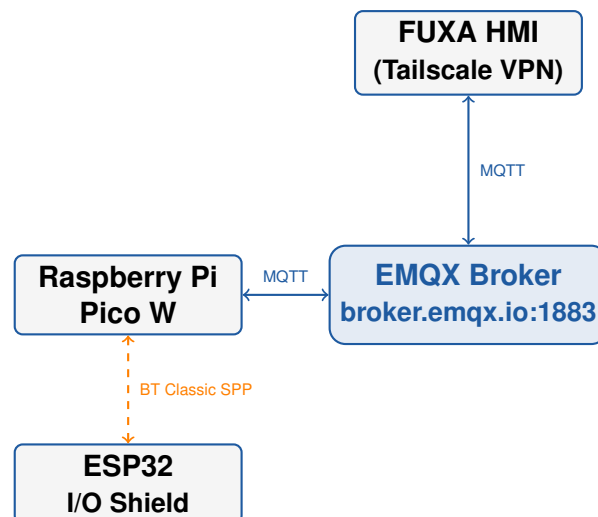
Core Deliverable (Demoed 02 Jul 2026)

- Bidirectional control of Pico W actuators (NeoPixels, buzzer, relay, servo) from FUXA and physical buttons.
- ESP32 sensor data (LDR, potentiometer, 3 LEDs, relay, buzzer) forwarded via BT Classic → Pico W → MQTT → FUXA.
- DHT11 temperature and humidity displayed as live gauges, logged to FUXA database, plotted as trend chart.
- Potentiometer on ESP32 controls Pico W servo smoothly (1 %/step, 0–180°).
- LDR-based automatic LED control with 10-sample moving-average filtering.
- MQTT Last Will and Testament for online/offline status.
- FUXA accessible remotely via Tailscale on mobile data.

System Architecture

Communication Topology

The system uses a **star topology** with the public EMQX broker as the central hub. A star topology was chosen because it allows any node to be added or removed independently, simplifies routing, and decouples the ESP32 from MQTT complexity.



Key design decision: The ESP32 does *not* connect to MQTT directly. All ESP32 sensor data is forwarded to the Pico W via Bluetooth Classic SPP. The Pico W acts as the sole MQTT gateway, managing all retained topics and LWT. This keeps the ESP32 firmware simple and prevents duplicate MQTT client IDs.

Hardware Nodes

Table 1: Hardware components per node

Node	Board	Peripherals	Link
Pico W	Joy-IT XPLODER	4× NeoPixel (GP1), DHT11 (GP0), SG90 servo (GP7), ST7735 TFT, buzzer (GP27), relay (GP28), 4× buttons	WiFi + BT Classic server
ESP32	DevKit + I/O Shield	LDR (GPIO32), potentiometer (GPIO33), 3× LEDs, 3× buttons, buzzer (GPIO18), relay (GPIO25)	BT Classic client
HMI	Laptop / phone	FUXA v1.3.3, SQLite log, trend charts	Tailscale VPN

MQTT Topic Architecture

Namespace and Broker

Topics are organised under two namespaces: `pico/` for Pico W entities and `esp32/` for ESP32 entities forwarded by the Pico W. The university broker (141.69.95.10:1883) was considered but the public EMQX broker was selected because the project runs on real hardware only (no Wokwi gateway is needed) and the university broker was not reachable from home networks during the unsupervised buffer phase.

Topic Map

Table 2: MQTT topic map

Topic	Dir.	Description
<i>Pico W — Commands (FUXA → Pico)</i>		
pico/led/cmd	↓	NeoPixel ON/OFF (0/1)
pico/buzzer/cmd	↓	Pico buzzer ON/OFF
pico/relay/cmd	↓	Pico relay ON/OFF
pico/disco/cmd	↓	Disco mode ON/OFF
<i>Pico W — Status (Pico → FUXA)</i>		
pico/dht/temp	↑	Temperature in °C
pico/dht/humidity	↑	Relative humidity %
pico/led/status	↑	ON / OFF
pico/relay/status	↑	ON / OFF
pico/buzzer/status	↑	ON / OFF
pico/servo/status	↑	Angle in degrees
pico/ldr/status	↑	DARK / BRIGHT
pico/wifi/status	↑	1 = online (retained, LWT = 0)
pico/status	↑	JSON object (all states)
<i>ESP32 — Commands (FUXA → Pico → BT → ESP32)</i>		
esp32/led1/cmd	↓	ESP32 LED1 (0/1)
esp32/led2/cmd	↓	ESP32 LED2 (0/1)
esp32/led3/cmd	↓	ESP32 LED3 (0/1)
esp32/relay/cmd	↓	ESP32 relay (0/1)
esp32/buzzer/cmd	↓	ESP32 buzzer (0/1)
<i>ESP32 — Status (ESP32 → BT → Pico → FUXA)</i>		
esp32/led1/status	↑	0 / 1
esp32/led2/status	↑	0 / 1
esp32/led3/status	↑	0 / 1
esp32/relay/status	↑	ON / OFF
esp32/buzzer/status	↑	ON / OFF
esp32/ldr/status	↑	DARK / BRIGHT

Reliability Features

- **Last Will & Testament:** Pico sets pico/wifi/status = 0 (retained) as its LWT; publishes 1 on connect. FUXA shows an online/offline indicator.
- **Retained topics:** pico/wifi/status is retained so any reconnecting subscriber immediately receives the last known state.
- **WiFi & MQTT watchdog:** Pico checks WiFi every 3 s and MQTT connection every 3 s, reconnecting automatically.
- **BT reconnect:** ESP32 detects SPP disconnection and retries connect("PicoW-BT") continuously in the main loop.

Key Features

Physical TFT HMI with Button Navigation

The 1.8 inch ST7735 TFT shows a two-column *SENSOR / STATUS* table with seven rows (Buzzer, Relay, LED, Servo, Temp, Humid, LDR). Navigation:

- **GP10** — row up
- **GP14** — row down
- **GP11** — toggle preview value (displayed immediately but not yet executed)
- **GP15** — long-press 2 s to confirm and execute the previewed change

A filled green circle next to each row indicates the component is physically ON; an empty circle indicates OFF. The WiFi SSID is displayed in a cyan status bar at the bottom. Partial row redraws (`redrawRow(i)`) eliminate full-screen flicker.

The boot sequence clears the screen before each phase (WiFi, MQTT) and holds each status screen for 1.5 s, producing clean transitions.

Smooth Servo Control

The SG90 servo is attached with an extended PWM range (`attach(pin, 500, 2500)`) to reach true 0–180°. A non-blocking incremental mover runs every 15 ms in the main loop, advancing `currentServoPos` by one degree per step toward `targetServoPos`. This eliminates the voltage-spike wobble caused by direct `write()` calls from MQTT callbacks.

LDR Automatic Lighting with Moving-Average Filter

The ESP32 computes a 10-sample circular-buffer moving average of the LDR ADC reading (GPIO 32). Publishing occurs only when the derived state changes (DARK/BRIGHT), reducing MQTT traffic. On the Pico W, automatic LED activation is tracked with a `ledAutoOn` flag so that manual overrides from FUXA or the TFT HMI are not overwritten by subsequent LDR readings.

Temperature-Triggered ESP32 Relay

The Pico publishes the DHT11 temperature to `pico/dht/temp` every 10 s. The ESP32 subscribes and activates its relay (simulating an air-conditioner) when temperature exceeds 30 °C, and deactivates it when it returns to 30 °C or below. FUXA reflects the relay state in real time via the `esp32/relay/status` topic.

Bidirectional LED Synchronisation (SYNC Pattern)

ESP32 LEDs can be toggled by physical buttons or commanded from FUXA. Every state change — whether from a button press or a FUXA command — triggers an `ESP_LEDn_SYNC` message over BT, which the Pico W republishes simultaneously to both `esp32/ledn/status` *and* `esp32/ledn/cmd`. FUXA switch widgets linked to the `cmd` topic therefore reflect the physical state regardless of who initiated the change, without a feedback loop.

Disco Mode

A non-blocking colour-cycling function steps the four NeoPixels through six hues at 200 ms per step while sounding the buzzer at a matching frequency, using `millis()` gating so MQTT polling and BT reads continue uninterrupted. Disco runs until FUXA sends `pico/disco/cmd = 0`.

Remote Access via Tailscale

FUXA runs on a laptop; Tailscale assigns a stable private IP reachable from any network without port forwarding. During the demo the dashboard was accessed from a phone on mobile data with sub-second response latency.

FUXA Dashboard

The FUXA v1.3.3 *Integrated HOME Operation* dashboard provides:

- **Pico panel** — toggle switches for LED, buzzer, relay, disco; red/green status indicators.
- **ESP32 panel** — toggle switches for LED1/2/3, relay, buzzer; status indicators.
- **Servo & LDR cloud widget** — current servo angle in degrees; DARK/BRIGHT text.
- **Temperature & Humidity gauges** — arc gauges with numeric readout (live).
- **Trend chart** — continuous time-series of Pico temperature (red) and humidity (blue), stored in FUXA's SQLite database.
- **Data log table** — timestamped rows with columns: Temperature, Humidity, LDR, Servo angle.

Pin Assignments

Table 3: Pico W (Joy-IT XPLOER) pin assignments

Peripheral	GPIO	Note
NeoPixel ×4	GP1	NEO_GRB, 800 kHz
DHT11	GP0	
Servo (SG90)	GP7	PWM 500–2500 μs
TFT CS	GP17	ST7735 1.8 inch
TFT DC	GP3	
TFT RST	GP6	
TFT Backlight	GP2	
Buzzer	GP27	tone()
Relay	GP28	
Button UP	GP10	INPUT_PULLUP
Button DOWN	GP14	INPUT_PULLUP
Button TOGGLE	GP11	INPUT_PULLUP
Button CONFIRM	GP15	INPUT_PULLUP, long-press 2 s

Table 4: ESP32 I/O Shield pin assignments

Peripheral	GPIO	Note
Potentiometer	GPIO33	ADC, 0–4095 → 0–180°
LDR	GPIO32	ADC, 10-sample moving avg
Button 1	GPIO26	toggles LED1
Button 2	GPIO27	toggles LED2
Button 3	GPIO5	toggles LED3
LED 1	GPIO15	
LED 2	GPIO2	
LED 3	GPIO4	
Relay	GPIO25	
Buzzer	GPIO18	tone()

Software Architecture

Pico W Firmware

The Pico W firmware (Arduino/C++, `pico_final.ino`) uses a single non-blocking main loop with `millis()` gating for all time-critical tasks:

- **BT receive:** `SerialBT.readStringUntil('\n')` parses KEY:VALUE frames from the ESP32.
- **MQTT:** `ArduinoMqttClient` with `onMqttMessage()` callback; WiFi and MQTT watchdogs every 3 s.
- **Servo:** Non-blocking 1°/15 ms incremental mover.
- **Disco:** Non-blocking colour step every 200 ms.
- **TFT HMI:** Partial row redraws only to avoid full-screen flicker.
- **DHT11:** Polled every 10 s; publishes integer temperature and humidity, then a full JSON status object.

ESP32 Firmware

The ESP32 firmware (`esp32_final.ino`, Arduino/C++) operates as a pure BT master with no MQTT dependency:

- **BT Master:** `BluetoothSerial` in master mode; connects to “PicoW-BT” by name and auto-reconnects.
- **LDR:** 10-sample circular buffer; publishes LDR:DARK or LDR:BRIGHT only on state change.
- **POT:** Raw ADC (0–4095) sent every 300 ms; Pico maps to servo angle.
- **Button debounce:** Rising-edge detection with 300 ms lockout plus 50 ms confirmation.
- **LED sync:** Every button toggle sends `ESP_LEDn_SYNC` immediately after the physical state change.

Stretch Goals Achieved

Table 5: Stretch goals relative to original B3 card

Feature	Status	Implementation
Second hardware node (ESP32)	✓	I/O shield, BT Classic bridge
Bluetooth Classic SPP link	✓	SerialBT (server) / BluetoothSerial (client)
Smooth servo (no wobble)	✓	1 %15 ms non-blocking mover
LDR auto-lighting with filter	✓	10-sample moving average
Temp-triggered relay (ESP32)	✓	Relay ON if temp > 30 °C
FUXA data logging + trend chart	✓	SQLite log, time-series graph
Remote access via Tailscale	✓	FUXA reachable from any network
Physical TFT HMI navigation	✓	4-button, dot indicators, SSID bar
Bidirectional LED sync	✓	SYNC message pattern
LWT / online-offline status	✓	Retained pico/wifi/status

Reflection

What Worked Well

The **BT Classic SPP bridge** proved more reliable than BLE for this use case: pairing survives reboots, throughput comfortably handles 300 ms sensor frames, and the `KEY:VALUE\n` text protocol is trivially debuggable with a serial monitor on either side.

The **SYNC message pattern** cleanly resolved the dual-control problem (physical button + FUXA switch) without a feedback loop. The `ledAutoOn` flag similarly decoupled automatic LDR lighting from manual overrides.

FUXA's combination of toggle switches (write to `cmd` topics) and status indicators (read from `status` topics), linked to separate tags, provided a clean bidirectional control flow that matched the MQTT model naturally.

Challenges

- **TLS on Nano 33 IoT:** The original B3 path required uploading HiveMQ root certificates to the WiFiNINA module. The upload tool failed repeatedly with "Upload failed" errors. Switching to the public EMQX broker on port 1883 resolved connectivity for the Pico W phase.
- **BLE on Pico W:** ArduinoBLE is incompatible with the RP2040 5.6.0 board package; BTstackLib GATT notifications have a known upstream bug (arduino-pico issue #2231) where `gattCharacteristicNotification` is never triggered. BT Classic SPP was chosen as a robust alternative.
- **Servo wobble:** Direct `myServo.write()` from inside MQTT callbacks caused PWM glitches. Moving to a non-blocking incremental mover in the main loop

eliminated the wobble completely.

- **FUXA switch sync:** FUXA switch widgets lock their visual state after sending a command and ignore subsequent incoming values on the same topic. Resolved by routing all physical button presses through a SYNC republish to the `cmd` topic.

Future Work

- Migrate to the university broker (141.69.95.10:1883) with `iem2026` token authentication as per course convention.
- Add MQTT Discovery so the system also appears natively in Home Assistant, completing the original B3 vision.
- Implement persistent state recovery on Pico W reboot by subscribing to retained `cmd` topics on connect.

Appendix A: Libraries Used

Library	Board	Purpose
ArduinoMqttClient	Pico W	MQTT client
ArduinoJson	Pico W	JSON status serialisation
Adafruit NeoPixel	Pico W	WS2812 LED strip
Adafruit ST7735	Pico W	TFT display driver
DHT sensor library	Pico W	DHT11 temp/humidity
Servo (rp2040)	Pico W	SG90 servo control
SerialBT	Pico W	BT Classic SPP server
BluetoothSerial	ESP32	BT Classic SPP master

Appendix B: Submission Contents

- `pico_final.ino` — commented Pico W firmware
- `esp32_final.ino` — commented ESP32 firmware
- `fuxa_homeautomation1.json` — FUXA project export
- `report.pdf` — this document